



Queues

A Friendly Guide to Understanding Queues

2026 ACM Handout Series

100 Days Of Code

Abstract: This handout covers queues from fundamentals to advanced applications. You'll learn FIFO semantics, circular buffers, priority queues, and deque patterns used in operating systems, BFS, and competitive programming. Includes detailed diagrams, C code, and real-world analogies.

© 2026 ACM Handout Series
100 Days Of Code
All rights reserved.

Contents

S.No	Topic	Page
1	Motivation - Why Queues Exist	1
2	Foundation - Core Concepts	2
3	Visualization - How It Looks	5
4	Operations - What You Can Do	7
5	Algorithm Patterns - CP-Focused	8
6	Implementation - Code Examples in C	13
7	Tricky Problems & Debugging	17
8	Types of Queues	22
9	Theory - Deep Concepts	27
10	Memory Allocation - Stack vs Heap	28
11	Practice - Problem Ladder	29
12	Reference - Quick Look	31
13	Competitive Programming Tips	32
14	More Tricky Debugging Problems	33
15	End of Handout	38

1 MOTIVATION - Why Queues Exist

1.1 The Problem Arrays and Linked Lists Can't Solve Alone

Consider a print spooler. Ten documents are submitted by ten different processes. The only fair policy is first-come, first-served. If you model this with a plain array and remove from the front by shifting every remaining element, each removal costs $O(n)$ moves. A spooler processing thousands of jobs would degrade to a quadratic-time bottleneck.

A linked list solves the $O(n)$ removal from the front (just move the head pointer— $O(1)$), but insertion at the tail is $O(n)$ unless you track a separate tail pointer. And crucially, nothing about the raw 'struct Node' prevents a caller from inserting in the middle, reading an arbitrary element, or forgetting to maintain the tail—violating the FIFO contract the rest of the program depends on.

What C Gives You: Raw arrays (contiguous, $O(1)$ index access, $O(n)$ shift) and raw linked lists (scattered, *can* be FIFO, but nothing enforces it). Neither one has a built-in fairness guarantee.

Definition: A queue is a linear data structure where elements are inserted at one end (rear) and removed from the other (front)—First-In, First-Out (FIFO).

The Core Insight: A queue wraps a raw array or linked list in a small 'struct' plus a disciplined interface (enqueue, dequeue, peek, is_empty) that guarantees the first element inserted is the first element removed, with both operations running in $O(1)$.

1.2 Arrays vs Linked Lists vs Queues - The Trade-off

Aspect	Array	Linked List	Queue ADT
FIFO enforced?	No	No	Yes
Enqueue (insert rear)	$O(1)$ at end	$O(1)$ with tail ptr	$O(1)$
Dequeue (remove front)	$O(n)$ shift	$O(1)$ at head	$O(1)$
Memory	Contiguous	Scattered	Contiguous or scattered
Cache locality	Excellent	Poor	Excellent (circular variant)
Risk of silent misuse	High	High	Low

1.3 When to Use a Queue

- When you need to process items in the order they arrived (fairness / FIFO)
- Breadth-First Search over trees or graphs (level-order traversal)
- Buffering between fast producer and slow consumer (I/O, print spoolers, message queues)
- Round-robin scheduling (each task gets a time slice, then re-enqueues)
- Sliding window over a stream (deque specialization)

2 FOUNDATION - Core Concepts

2.1 The Anatomy of a Queue

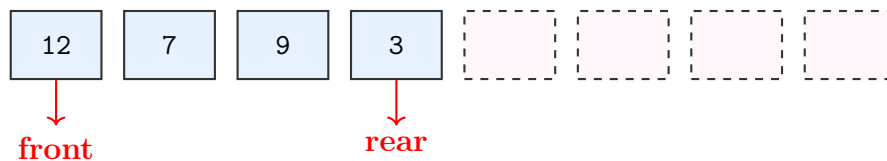
A queue needs exactly two coordinates: **where the next dequeue comes from** (front) and **where the next enqueue goes** (rear).

Array-Based Queue Structure

```

1 #define CAPACITY 8
2
3 typedef struct {
4     int data[CAPACITY];
5     int front;    // Index of current front element
6     int rear;     // Index of last inserted element
7     int count;    // Number of elements currently stored
8 } ArrayQueue;

```

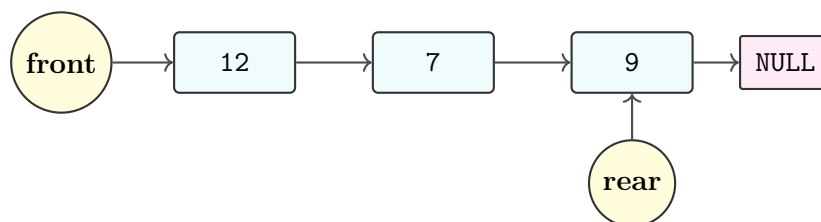


Linked-List-Based Queue Structure

```

1 typedef struct Node {
2     int data;
3     struct Node *next;
4 } Node;
5
6 typedef struct {
7     Node *front; // Pointer to front node (NULL if empty)
8     Node *rear;  // Pointer to rear node (NULL if empty)
9     int count;
10 } LinkedList;

```



2.2 The Critical Concept: The Circular Wrapping

The single most confusing idea in an array-based queue: **rear must wrap around to index 0 once it reaches the end**. Otherwise, the queue reports "full" even when empty slots exist at the front.

Wrong (linear, non-circular):

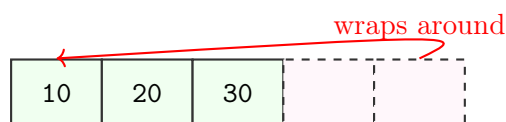
```
1 int enqueue_wrong(ArrayQueue *q, int value) {  
2     if (q->rear == CAPACITY - 1)  
3         return -1; // WRONG! Slots at front may be free!  
4     q->rear++;  
5     q->data[q->rear] = value;  
6     q->count++;  
7     return 0;  
8 }
```

Correct (circular, modulo):

```
1 int enqueue_correct(ArrayQueue *q, int value) {  
2     if (q->count == CAPACITY)  
3         return -1; // Genuinely full  
4     q->rear = (q->rear + 1) % CAPACITY;  
5     q->data[q->rear] = value;  
6     q->count++;  
7     return 0;  
8 }
```

WARNING: `front == rear` is ambiguous—does it mean empty or full? Always track count explicitly to resolve this.

The ‘`% CAPACITY`’ is what turns a linear array into a **ring**: incrementing past the last index wraps back to 0, so freed front slots are always reachable by rear.



2.3 Queue Terminology

Term	Definition
Enqueue	Insert a new element at the rear of the queue
Dequeue	Remove and return the element at the front of the queue
Front	The end from which elements are removed
Rear	The end at which elements are inserted
Peek	Return the front element without removing it
FIFO	First-In, First-Out - the ordering discipline
Overflow	Attempted enqueue on a full, fixed-capacity queue
Underflow	Attempted dequeue on an empty queue
Capacity	Maximum number of elements a fixed-size queue can hold
Circular Buffer	Array-based queue using modulo wrapping
Deque	Double-Ended Queue - insertion and removal at both ends
Priority Queue	Queue where the element with highest priority (here, smallest value) is removed first

2.4 Syntax Across Languages

Language	Syntax
C (array)	<code>int data[CAPACITY], front, rear, count;</code>
C (linked)	<code>struct Node { int data; struct Node* next; };</code>
C++	<code>std::queue<int> q;</code>
Java	<code>Queue<Integer> q = new LinkedList<>();</code>
Python	<code>from collections import deque</code>

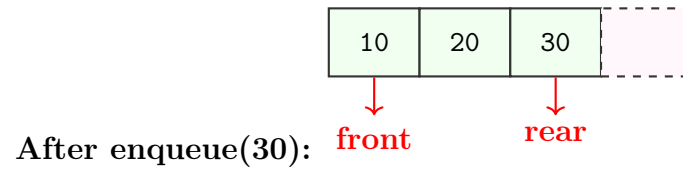
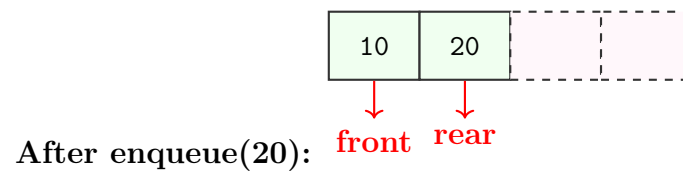
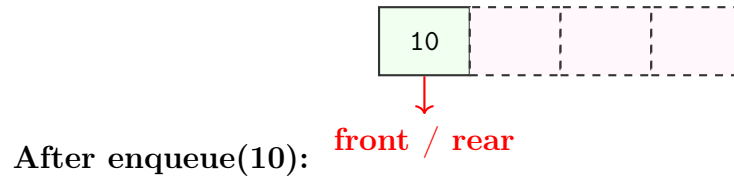
2.5 Real-Life Analogies

1. **The Bank Teller Line:** Customers join at the back and are served from the front. Nobody is served out of turn—the exact FIFO discipline that enqueue/dequeue enforce.
2. **Conveyor Belt Buffer:** One machine produces faster than the next can consume. The queue between them absorbs the mismatch. Parts accumulate in the buffer during a production burst and are consumed from the front at the slower machine's pace.
3. **Circular Parking Garage (circular variant):** Cars enter one gate and exit another. Once a car leaves, its spot is immediately available for the next arriving car, even though physically it's "behind" newer cars. Exactly like `(rear + 1) % CAPACITY`.

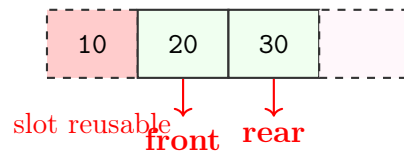
3 VISUALIZATION - How It Looks

3.1 Visual Representation of a Circular Queue

3 Enqueues (10, 20, 30) on a 4-slot Queue

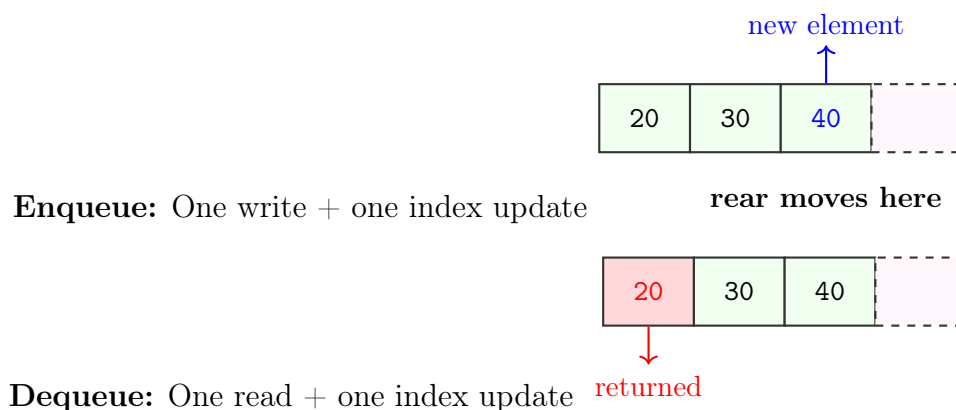


After dequeue() returns 10:



3.2 Why Access is $O(1)$ - The Visual Proof

Every operation touches only a constant number of elements, regardless of queue size.



Key Point: Whether the queue holds 10 elements or 10 million, enqueue and dequeue still run in constant time because they never iterate over elements. This is the fundamental advantage over the naive shifting array approach where dequeue costs $O(n)$.

Proof: enqueue performs exactly `rear = (rear + 1) % CAP; data[rear] = value; count++;` — three operations, none of which loop. Dequeue is symmetric. Since instruction count never depends on count, asymptotic cost is constant: $O(1)$.

4 OPERATIONS - What You Can Do

4.1 Operations Complexity Table

Operation	Time	Reason
Enqueue (insert rear)	$O(1)$	One write, update rear, increment count
Dequeue (remove front)	$O(1)$	One read, update front, decrement count
Peek (view front)	$O(1)$	Direct array access or front pointer
is_empty	$O(1)$	Compare count to 0
is_full	$O(1)$	Compare count to capacity
Search (find value)	$O(n)$	Must traverse entire queue

Pattern: The core queue operations (enqueue, dequeue, peek) are all $O(1)$. The price for this speed is that you can only access the **front** element—no random access, no middle insertion.

4.2 The Golden Rules of Queues

1. A queue has exactly **two moving ends** — front (dequeue) and rear (enqueue). If you only track one, it's not a queue.
2. Track **count explicitly**. `front == rear` is ambiguous between empty and full.
3. Always use **modulo wrapping**: `(index + 1) % CAPACITY` — not subtraction, not manual if-statements.
4. Never dereference front/rear without checking `is_empty()` first.
5. Every malloc in a linked queue has **exactly one matching free**.
6. BFS always uses a queue, never a stack — swapping in a stack silently breaks the shortest-path guarantee.

5 ALGORITHMS/PATTERNS - CP-Focused

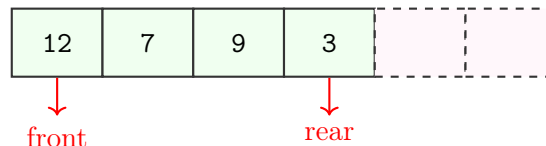
5.1 Pattern 1: The Bounded Circular Queue

When to use: Fixed-capacity FIFO buffering, embedded systems, network packet ring buffers.

The Template:

```
1 void init(ArrayQueue* q) {
2     q->front = 0;
3     q->rear = -1;
4     q->count = 0;
5 }
6
7 int enqueue(ArrayQueue* q, int value) {
8     if (q->count == CAPACITY) return -1; // Full
9     q->rear = (q->rear + 1) % CAPACITY;
10    q->data[q->rear] = value;
11    q->count++;
12    return 0;
13 }
14
15 int dequeue(ArrayQueue* q, int* out) {
16     if (q->count == 0) return -1; // Empty
17     *out = q->data[q->front];
18     q->front = (q->front + 1) % CAPACITY;
19     q->count--;
20     return 0;
21 }
```

Visual:

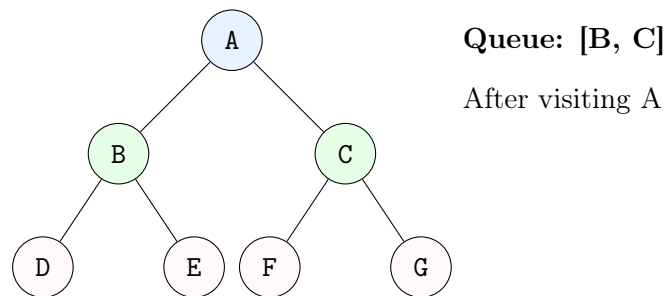


5.2 Pattern 2: BFS - Breadth-First Search

When to use: Shortest path in unweighted graph, level-order tree traversal, minimum steps in a grid.

Analogy: Ripples in a pond. When you drop a stone, the ripples expand outward one layer at a time. BFS does exactly this—visit all neighbors first, then their neighbors.

Visual:



```

1  #define MAXV 100
2  int adj[MAXV][MAXV];
3  int n_vertices;
4
5  void bfs(int start, int dist[MAXV]) {
6      int queue[MAXV], front = 0, rear = -1, count = 0;
7      int visited[MAXV] = {0};
8
9      // Enqueue start (guard against full queue)
10     if (count < MAXV) {
11         rear = (rear + 1) % MAXV;
12         queue[rear] = start;
13         count++;
14     }
15     visited[start] = 1;
16     dist[start] = 0;
17
18     while (count > 0) {
19         int u = queue[front];
20         front = (front + 1) % MAXV;
21         count--;
22
23         for (int v = 0; v < n_vertices; v++) {
24             if (adj[u][v] && !visited[v]) {
25                 visited[v] = 1;

```

```
26         dist[v] = dist[u] + 1;
27         if (count < MAXV) {
28             rear = (rear + 1) % MAXV;
29             queue[rear] = v;
30             count++;
31         }
32     }
33 }
34 }
35 }
```

Time: $O(V + E)$ (each vertex enqueued/dequeued once, each edge examined once).

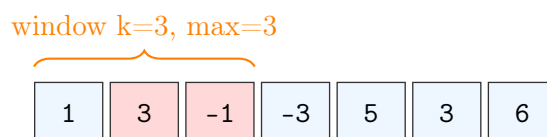
Variations: Multi-source BFS (enqueue multiple start nodes first), 0-1 BFS (use a deque: push 0-weight edges to the front, 1-weight edges to the back; the deque maintains a sorted distance order automatically).

5.3 Pattern 3: Monotonic Queue

When to use: You need the maximum (or minimum) of every sliding window in $O(n)$ total, not $O(n \cdot k)$ from a naive per-window scan.

The Idea: Maintain a deque of indices whose values are in decreasing order. The front is always the current window's maximum.

Visual:



```
1 void slidingWindowMax(int arr[], int n, int k, int res[]) {
2     if (n < k || n == 0) return;
3     int dq[n], front = 0, back = -1, ri = 0;
4
5     for (int i = 0; i < n; i++) {
6         // Remove indices outside window from front
7         if (front ≤ back && dq[front] ≤ i - k)
8             front++;
9
10        // Maintain decreasing order: pop smaller from back
11        while (front ≤ back && arr[dq[back]] ≤ arr[i])
12            back--;
13
14        dq[++back] = i;
15
16        // Front of deque is max of current window
17        if (i ≥ k - 1)
18            res[ri++] = arr[dq[front]];
19    }
20 }
```

Time: $O(n)$ overall — each index pushed and popped at most once.

5.4 Pattern 4: Sliding Window (Fixed-Size)

When to use: Computing running statistics over every contiguous window of size k in one $O(n)$ pass.

```
1 int maxSlidingWindowSum(int arr[], int n, int k) {  
2     if (n < k) return -1;  
3  
4     int windowSum = 0;  
5     for (int i = 0; i < k; i++) windowSum += arr[i];  
6     int maxSum = windowSum;  
7  
8     for (int i = k; i < n; i++) {  
9         windowSum += arr[i] - arr[i - k]; // Slide window  
10        if (windowSum > maxSum) maxSum = windowSum;  
11    }  
12    return maxSum;  
13 }
```

Time: $O(n)$, space $O(1)$.

6 IMPLEMENTATION - Code Examples in C

6.1 Complete Circular Array Queue

```
1  #include <stdio.h>
2
3  #define CAPACITY 5
4
5  typedef struct {
6      int data[CAPACITY];
7      int front;
8      int rear;
9      int count;
10 } CircularQueue;
11
12 void init(CircularQueue *q) {
13     q->front = 0;
14     q->rear = -1;
15     q->count = 0;
16 }
17
18 int isEmpty(const CircularQueue *q) { return q->count == 0; }
19 int isFull(const CircularQueue *q) { return q->count == CAPACITY; }
20
21 int enqueue(CircularQueue *q, int value) {
22     if (isFull(q)) return -1;
23     q->rear = (q->rear + 1) % CAPACITY;
24     q->data[q->rear] = value;
25     q->count++;
26     return 0;
27 }
28
29 int dequeue(CircularQueue *q, int *out) {
30     if (isEmpty(q)) return -1;
31     *out = q->data[q->front];
32     q->front = (q->front + 1) % CAPACITY;
33     q->count--;
34     return 0;
35 }
36
```

```
37 int peek(const CircularQueue *q, int *out) {
38     if (isEmpty(q)) return -1;
39     *out = q->data[q->front];
40     return 0;
41 }
42
43 void print(const CircularQueue *q) {
44     printf("[ ");
45     for (int i = 0, idx = q->front; i < q->count; i++) {
46         printf("%d ", q->data[idx]);
47         idx = (idx + 1) % CAPACITY;
48     }
49     printf("] (count=%d)\n", q->count);
50 }
```

6.2 Complete Linked-List Queue

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct Node {
5      int data;
6      struct Node *next;
7  } Node;
8
9  typedef struct {
10     Node *front;
11     Node *rear;
12     int count;
13 } LinkedListQueue;
14
15 void init(LinkedListQueue *q) {
16     q->front = NULL;
17     q->rear = NULL;
18     q->count = 0;
19 }
20
21 int isEmpty(const LinkedListQueue *q) { return q->count == 0; }
22
23 int enqueue(LinkedListQueue *q, int value) {
24     Node *n = (Node*)malloc(sizeof(Node));
25     if (n == NULL) return -1;
26     n->data = value;
27     n->next = NULL;
28
29     if (q->rear == NULL) {
30         q->front = n;
31         q->rear = n;
32     } else {
33         q->rear->next = n;
34         q->rear = n;
35     }
36     q->count++;
37     return 0;
38 }
```

```
39
40 int dequeue(LinkedQueue *q, int *out) {
41     if (isEmpty(q)) return -1;
42     Node *old = q->front;
43     *out = old->data;
44     q->front = old->next;
45     if (q->front == NULL) q->rear = NULL;
46     free(old);
47     q->count--;
48     return 0;
49 }
50
51 void destroy(LinkedQueue *q) {
52     Node *cur = q->front;
53     while (cur != NULL) {
54         Node *next = cur->next;
55         free(cur);
56         cur = next;
57     }
58     q->front = NULL;
59     q->rear = NULL;
60     q->count = 0;
61 }
```

7 TRICKY PROBLEMS & DEBUGGING

7.1 Problem 1: The Overflow Bug

Scenario: Writing past a full circular buffer silently overwrites dequeued data.

Bug: No capacity check before writing.

WRONG CODE:

```
1 void enqueue(CircularQueue *q, int val) {  
2     q->rear = (q->rear + 1) % CAPACITY;  
3     q->data[q->rear] = val;    // Overwrites oldest unread data!  
4     q->count++;  
5 }
```

CORRECT CODE:

```
1 int enqueue(CircularQueue *q, int val) {  
2     if (q->count == CAPACITY) return -1;  
3     q->rear = (q->rear + 1) % CAPACITY;  
4     q->data[q->rear] = val;  
5     q->count++;  
6     return 0;  
7 }
```

Key Lesson: Always check `is_full()` before writing. Let the caller detect and react to rejection.

7.2 Problem 2: The Underflow Bug

Scenario: Dequeuing from an empty queue returns garbage and corrupts count.

Bug: No emptiness check before reading.

WRONG CODE:

```
1 int dequeue(CircularQueue *q) {
2     int val = q->data[q->front];
3     q->front = (q->front + 1) % CAPACITY;
4     q->count--;           // count becomes -1 on empty queue!
5     return val;          // Returns garbage data!
6 }
```

CORRECT CODE:

```
1 int dequeue(CircularQueue *q, int *out) {
2     if (q->count == 0) return -1;
3     *out = q->data[q->front];
4     q->front = (q->front + 1) % CAPACITY;
5     q->count--;
6     return 0;
7 }
```

Key Lesson: Underflow doesn't just return bad data once—it corrupts count so every future call misbehaves.

7.3 Problem 3: The Modulo Arithmetic Mistake

Scenario: Forgetting the modulo when advancing rear leads to out-of-bounds writes.

Bug: No wraparound.

WRONG CODE:

```
1 int enqueue(CircularQueue *q, int value) {
2     if (q->count == CAPACITY) return -1;
3     q->rear = q->rear + 1;           // BUG! rear can exceed CAPACITY-1
4     q->data[q->rear] = value;       // OUT-OF-BOUNDS WRITE!
5     q->count++;
6     return 0;
7 }
```

CORRECT CODE:

```
1 int enqueue(CircularQueue *q, int value) {
2     if (q->count == CAPACITY) return -1;
3     q->rear = (q->rear + 1) % CAPACITY; // Always wraps correctly
4     q->data[q->rear] = value;
5     q->count++;
6     return 0;
7 }
```

Key Lesson: Default to the `% CAPACITY` form everywhere an index advances. Never write the wraparound as a manual if.

7.4 Problem 4: The NULL Dereference (Linked Queue)

Scenario: Dequeuing from an empty linked queue crashes with a segfault.

Bug: Dereferencing front without checking for NULL.

WRONG CODE:

```
1 int dequeue(LinkedList *q, int *out) {
2     *out = q->front->data;           // CRASH if queue is empty!
3     Node *old = q->front;
4     q->front = q->front->next;
5     free(old);
6     q->count--;
7     return 0;
8 }
```

CORRECT CODE:

```
1 int dequeue(LinkedList *q, int *out) {
2     if (q->front == NULL) return -1;
3     Node *old = q->front;
4     *out = old->data;
5     q->front = old->next;
6     if (q->front == NULL) q->rear = NULL; // Queue is now empty
7     free(old);
8     q->count--;
9     return 0;
10 }
```

Key Lesson: In a linked structure, "empty" means a NULL pointer. C never stops you from dereferencing one—the check must be explicit.

7.5 Problem 5: The Memory Leak (Linked Queue)

Scenario: Dequeue advances front pointer but never frees the old node.

Bug: Advancing front without freeing the old node.

WRONG CODE:

```
1 int dequeue(LinkedList *q, int *out) {  
2     if (q->front == NULL) return -1;  
3     *out = q->front->data;  
4     q->front = q->front->next;    // Memory leak! Old node still allocated  
5     q->count--;  
6     return 0;  
7 }
```

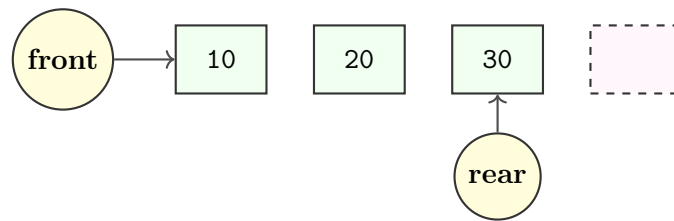
CORRECT CODE:

```
1 int dequeue(LinkedList *q, int *out) {  
2     if (q->front == NULL) return -1;  
3     Node *old = q->front;  
4     *out = old->data;  
5     q->front = old->next;  
6     if (q->front == NULL) q->rear = NULL;  
7     free(old);    // Must free the old node!  
8     q->count--;  
9     return 0;  
10 }
```

Key Lesson: In C, unlinking a node from a linked structure and freeing its memory are two separate steps. Every malloc needs a matching free on every code path.

8 TYPES OF QUEUES

Circular Queue



Characteristics:

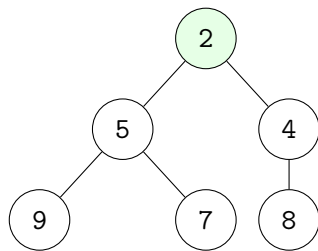
- Fixed capacity, array-backed, indices wrap via modulo
- Reuses freed front slots without shifting
- Default choice when a hard upper bound is known (embedded, ring buffers)
- Memory: $n * \text{sizeof}(\text{int}) + 3 \text{ ints (front, rear, count)}$

Best used for: Fixed-size task schedulers, audio/network I/O ring buffers, embedded systems.

Priority Queue (Binary Heap)

Characteristics:

- Element removed is always the one with **highest priority** (smallest value in a min-heap)
- Deliberately breaks FIFO—priority overrides insertion order
- Backed by a binary heap in a plain array
- $O(\log n)$ insertion, $O(\log n)$ removal-of-min, $O(1)$ peek



Min-heap invariant:
every parent $\leq$ its children.
Root (2) is always the next
element dequeued.

```

1  #define PQ_CAP 64
2  typedef struct {
3      int data[PQ_CAP];
4      int size;
5  } PriorityQueue;
6
7  void push(PriorityQueue *pq, int val) {
8      int i = pq->size++;
9      pq->data[i] = val;
10     // Sift up: swap with parent while smaller
11     while (i > 0 && pq->data[(i-1)/2] > pq->data[i]) {
12         int t = pq->data[i];
13         pq->data[i] = pq->data[(i-1)/2];
14         pq->data[(i-1)/2] = t;
15         i = (i-1)/2;
16     }
17 }
18
19 int pop(PriorityQueue *pq, int *out) {
20     if (pq->size == 0) return -1;
21     *out = pq->data[0];
22     pq->size--;
23     // decrement before indexing
24     // into data
25     pq->data[0] = pq->data[pq->size];

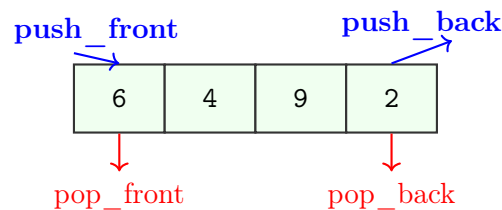
```

```
24 // Sift down: swap with smaller child
25 int i = 0;
26 while (1) {
27     int smallest = i, l = 2*i+1, r = 2*i+2;
28     if (l < pq->size && pq->data[l] < pq->data[smallest]) smallest = l;
29     if (r < pq->size && pq->data[r] < pq->data[smallest]) smallest = r;
30     if (smallest == i) break;
31     int t = pq->data[i]; pq->data[i] = pq->data[smallest]; pq->data[
        smallest] = t;
32     i = smallest;
33 }
34 return 0;
35 }
```

Deque (Double-Ended Queue)

Characteristics:

- Insertion and removal at **both ends**: push_front, push_back, pop_front, pop_back
- All operations $O(1)$
- More general than plain queue or stack—either can be built from a deque
- Powers the monotonic queue pattern (Pattern 3)



```

1  #define DQ_CAP 8
2  typedef struct {
3      int data[DQ_CAP];
4      int front, rear, count;
5  } Deque;
6
7  void init(Deque *dq) { dq->front = 0; dq->rear = -1; dq->count = 0; }
8
9  int push_back(Deque *dq, int v) {
10     if (dq->count == DQ_CAP) return -1;
11     dq->rear = (dq->rear + 1) % DQ_CAP;
12     dq->data[dq->rear] = v; dq->count++; return 0;
13 }
14
15 int push_front(Deque *dq, int v) {
16     if (dq->count == DQ_CAP) return -1;
17     dq->front = (dq->front - 1 + DQ_CAP) % DQ_CAP;
18     dq->data[dq->front] = v; dq->count++; return 0;
19 }
20
21 int pop_front(Deque *dq, int *out) {
22     if (dq->count == 0) return -1;
23     *out = dq->data[dq->front];
24     dq->front = (dq->front + 1) % DQ_CAP; dq->count--; return 0;
25 }

```

```
26
27 int pop_back(Deque *dq, int *out) {
28     if (dq->count == 0) return -1;
29     *out = dq->data[dq->rear];
30     dq->rear = (dq->rear - 1 + DQ_CAP) % DQ_CAP; dq->count--; return 0;
31 }
```

9 THEORY - Deep Concepts

9.1 Why the Circular Array?

A queue's real purpose—buffering, scheduling—is to run indefinitely, processing a bounded burst of pending work at any moment. A linear (non-wrapping) array would need to keep shifting its "used region" forward through memory as elements are dequeued from the front, eventually running off the end even though most of the array is empty. Two options: shift everything back down periodically (wasted $O(n)$ work) or wrap the index arithmetic ($O(1)$, no wasted work). Circular wrapping is simply the $O(1)$ answer.

9.2 Why Both Array and Linked Implementations?

- **Array-based (circular):** Wins when max size is known and cache locality matters. One contiguous block, one malloc for lifetime. Used in: embedded systems, audio buffers, network packet ring buffers.
- **Linked-list-based:** Wins when max size is unknown or highly variable. Grows one node at a time, no capacity check. Cost: one malloc per enqueue, one free per dequeue, plus poor cache locality.

9.3 Why Cache Locality Matters

Modern CPUs fetch memory in cache-line-sized chunks (typically 64 bytes). Walking a circular array touches memory the CPU has likely already prefetched. Walking a linked list means following a 'next' pointer to a malloc'd address that could be anywhere on the heap—each dereference is a potential cache miss, which on modern hardware can cost 100x the latency of an L1 cache hit.

9.4 Why a Priority Queue Uses a Heap (Not a Sorted Array)

A sorted array gives $O(1)$ min but $O(n)$ insertion. A binary heap gives up full sortedness—it only guarantees the root is the minimum—in exchange for $O(\log n)$ insertion. This is the correct trade for a priority queue: you almost never need the k -th smallest element, only "give me the current minimum, repeatedly, while other elements keep arriving."

10 MEMORY ALLOCATION - Stack vs Heap

Aspect	Stack	Heap
Allocation	Automatic	Manual (malloc)
Speed	Fast	Slower
Lifetime	Function scope	Until free()
Fixed capacity	Compile-time (#define)	Runtime (variable)
Risk	Stack overflow if large	Memory leak if not freed

Best Practice: Use stack-allocated arrays for queues with known, small-to-moderate capacity. Use heap allocation when capacity is runtime-dependent or when the queue must outlive its creating function. Note: heap-allocating the struct allocates the `data[]` array together with it in one block—no separate `malloc` for the array is needed.

Key insight: A circular array queue of adequate fixed capacity pays **zero allocation calls** after initialization. A linked queue processing a million enqueue/dequeue pairs pays a million `malloc` calls and a million `free` calls. This is the single biggest reason competitive programmers default to pre-allocated static arrays.

11 PRACTICE - Problem Ladder

11.1 Level 1: Fundamentals

- Implement a queue using two stacks
- Implement a circular array queue from scratch
- Implement a linked-list queue from scratch
- Reverse the first k elements of a queue

11.2 Level 2: Intermediate

- First unique character in a string (queue + frequency)
- Generate binary numbers from 1 to n using a queue
- Implement a stack using two queues
- Rotting oranges (multi-source BFS on grid)

11.3 Level 3: Advanced

- Sliding window maximum (monotonic deque)
- Shortest path in a binary matrix (BFS)
- Design a hit counter (queue-based time window)
- Task scheduler with cooldown (queue simulation)

Problem Hints - Stuck? Start Here

Queue using two stacks - hint: Push everything onto stack A. When you need to dequeue and stack B is empty, pour all of A into B first (this reverses the order exactly once).

First unique character - hint: Use a freq[256] array and a queue of characters. When a character repeats, mark it in freq and skip it during dequeue.

Rotting oranges - hint: Enqueue ALL initially rotten oranges before the BFS loop starts. Track minutes as the current BFS "distance" from the nearest source.

Sliding window max - hint: The deque should hold INDICES, not values, in decreasing order. Anything smaller than the newly arriving element can never be the max again—evict it from the back immediately.

12 REFERENCE - Quick Look

12.1 Complexity Table

Operation	Time	Space
Enqueue (insert rear)	$O(1)$	$O(1)$
Dequeue (remove front)	$O(1)$	$O(1)$
Peek (view front)	$O(1)$	$O(1)$
is_empty / is_full	$O(1)$	$O(1)$
Search	$O(n)$	$O(1)$

Cheat Sheet - The 7 Essential Queue Operations

Pattern	When to Use	Complexity
Circular Queue	Fixed-capacity FIFO, ring buffers	$O(1)$ per op
BFS / Level-Order	Shortest path, tree traversal	$O(V + E)$
Monotonic Queue	Sliding window max/min	$O(n)$
Sliding Window (sum)	Running sum over window of size k	$O(n)$
Priority Queue (Heap)	Always need highest-priority item	$O(\log n)$ per op
Deque	Double-ended operations	$O(1)$ per op
Multi-Source BFS	Multiple start points in a grid	$O(V + E)$

Remember these rules:

1. Always check empty/full before dequeue/enqueue
2. Use modulo for circular wrapping: $(i + 1) \% CAP$
3. Track count explicitly (never rely on `front == rear`)
4. In linked queues: every malloc needs a matching free
5. BFS needs a queue, not a stack
6. For negative modulo in deque operations: $(i - 1 + CAP) \% CAP$

13 COMPETITIVE PROGRAMMING TIPS

CP-Specific Tips for C

Faster I/O in C: Use `scanf/printf`. For very large inputs, use custom fast I/O with `getchar()/putchar()`.

1. **Prefer static/stack-allocated arrays over malloc'd ones.** Declare `int queue[MAX_N];` globally (zero-initialized, no stack overflow) rather than dynamic sizing.
2. **Fast I/O matters more than queue logic.** For 10^6+ integers, `scanf` can become the bottleneck. A custom integer reader using `getchar()` is the standard trick.
3. **Global arrays for large capacity.** A local `CircularQueue` with `data[1000000]` can overflow the stack. Use global or static storage instead.
4. **Avoid malloc/free in BFS hot loops.** Pre-allocate the queue array once, sized to the maximum number of vertices.
5. **Reset global arrays between test cases.** In multi-test-case problems, forgetting to re-init is one of the most common judge-failing bugs.
6. **Track size explicitly.** Using `front == rear` to infer empty/full is error-prone—especially under contest time pressure.

Common Mistakes in C

1. **Forgetting to free dequeued nodes (linked queue):** Every `malloc` needs a matching `free`.
2. **Forgetting to check `is_full()` before enqueue:** Silently overwrites the oldest undequeued data.
3. **Forgetting to check `is_empty()` before dequeue:** Returns garbage and corrupts count.
4. **Using non-wrapping index arithmetic:** `rear++` instead of `(rear+1) % CAP` causes out-of-bounds writes.
5. **Not updating both `front` and `rear` when queue goes empty:** In linked queues, forgetting to set `rear = NULL` after the last dequeue leaves a dangling tail pointer.
6. **Negative modulo in C:** `(-1) % CAP` is negative in C (unlike Python). Always use `(i - 1 + CAP) % CAP` for descending circular indices.
7. **Stack overflow from large local queue arrays:** Use global or static arrays for large capacities.

14 MORE TRICKY DEBUGGING PROBLEMS

Problem 6: The Tail Not Updated

Scenario: After dequeuing the last node in a linked queue, rear still points to freed memory.

Bug: Forgetting to set rear = NULL when the queue becomes empty.

WRONG CODE:

```
1 int dequeue(LinkedList *q, int *out) {
2     Node *old = q->front;
3     *out = old->data;
4     q->front = old->next;
5     free(old);
6     q->count--;
7     return 0;
8     // BUG: if this was the last node, q->rear still points
9     // to the now-freed node! Next enqueue corrupts the queue.
10 }
```

CORRECT CODE:

```
1 int dequeue(LinkedList *q, int *out) {
2     if (q->front == NULL) return -1;
3     Node *old = q->front;
4     *out = old->data;
5     q->front = old->next;
6     if (q->front == NULL) q->rear = NULL; // Queue is now empty
7     free(old);
8     q->count--;
9     return 0;
10 }
```

Problem 7: The Dangling Pointer After Dequeue

Bug: Reading node data after freeing.

WRONG CODE:

```
1 int dequeue(LinkedList *q, int *out) {
2     Node *old = q->front;
3     free(old);                // Freed too early!
4     *out = old->data;          // Reading freed memory = undefined behavior
5     q->front = old->next;
6     q->count--;
7     return 0;
8 }
```

CORRECT CODE:

```
1 int dequeue(LinkedList *q, int *out) {
2     if (q->front == NULL) return -1;
3     Node *old = q->front;
4     *out = old->data;          // Read data FIRST
5     q->front = old->next;
6     if (q->front == NULL) q->rear = NULL;
7     free(old);                // Then free
8     q->count--;
9     return 0;
10 }
```

Key Lesson: Always extract data from a node before freeing it. The order matters.

Problem 8: The Negative Modulo in C

Scenario: Decrementing a circular index without handling C's negative modulo.

Bug: Using $(i - 1) \% \text{CAP}$ directly—C's $\%$ gives negative results for negative operands.

WRONG CODE:

```
1 int pop_back(Deque *dq, int *out) {
2     *out = dq->data[dq->rear];
3     dq->rear = (dq->rear - 1) % DQ_CAP; // If rear was 0, this is (-1) %
        DQ_CAP
4     dq->count--; // which is -1 in C, NOT
        DQ_CAP - 1!
5     return 0;
6 }
```

CORRECT CODE:

```
1 int pop_back(Deque *dq, int *out) {
2     if (dq->count == 0) return -1;
3     *out = dq->data[dq->rear];
4     dq->rear = (dq->rear - 1 + DQ_CAP) % DQ_CAP; // +CAP ensures positive
        operand
5     dq->count--;
6     return 0;
7 }
```

Key Lesson: In C, $(-1) \% N$ is -1 , not $N-1$. Always add the capacity before applying modulo: $(i - 1 + \text{CAP}) \% \text{CAP}$.

Problem 9: The Off-by-One in Sliding Window

Scenario: Wrong indices in the sliding window loop skip or double-count elements.

Bug: Adding `arr[i]` but subtracting `arr[i-k-1]` instead of `arr[i-k]`.

WRONG CODE:

```
1 int maxSlidingWindowSum(int arr[], int n, int k) {
2     int sum = 0;
3     for (int i = 0; i < k; i++) sum += arr[i];
4     int max = sum;
5     for (int i = k; i < n; i++) {
6         sum += arr[i] - arr[i - k - 1]; // BUG: off by one!
7         if (sum > max) max = sum;
8     }
9     return max;
10 }
```

CORRECT CODE:

```
1 int maxSlidingWindowSum(int arr[], int n, int k) {
2     int sum = 0;
3     for (int i = 0; i < k; i++) sum += arr[i];
4     int max = sum;
5     for (int i = k; i < n; i++) {
6         sum += arr[i] - arr[i - k]; // Remove arr[i-k], add arr[i]
7         if (sum > max) max = sum;
8     }
9     return max;
10 }
```

Why: When the window slides from position to position+1, element `i-k` exits the window and element `i` enters.

Problem 10: BFS Without Queue Overflow Guard

Scenario: BFS queue overflows on a dense graph because the array size was calculated as MAXV instead of $\text{MAXV} * \text{MAXV}$.

Bug: Underestimating the maximum queue size needed for BFS.

WRONG CODE:

```
1 void bfs(int start) {
2     int queue[MAXV], front = 0, rear = -1, count = 0;
3     // BUG: In a complete graph, neighbor queue could hold up to
4     // MAXV-1 nodes. But if BFS visits all nodes, the queue might
5     // need to hold multiple levels simultaneously. In a dense graph
6     // this WRITES PAST queue[MAXV-1]!
7 }
```

CORRECT CODE:

```
1 // Portable C: no nested functions (GNU extension).
2 // Guard every enqueue with a capacity check.
3 #define enqueue_if_room(q, v, cap, cnt, rear) \
4     do { if ((cnt) < (cap)) {                \
5         (rear) = ((rear) + 1) % (cap);        \
6         (q)[rear] = (v);                      \
7         (cnt)++;                              \
8     } } while (0)
9
10 void bfs(int start) {
11     int queue[MAXV], front = 0, rear = -1, count = 0;
12
13     enqueue_if_room(queue, start, MAXV, count, rear);
14     // ... rest of BFS, using enqueue_if_room for every enqueue
15 }
```

Key Lesson: A BFS queue needs at most V elements for a graph with V vertices. Size your queue to MAXV —but always include bounds checks.

15 END OF HANDOUT

Congratulations! You've mastered Queues!

What you can do now:

- Implement circular array and linked-list queues from scratch in C
- Use 4 core algorithmic patterns: BFS, sliding window, monotonic queue, circular buffer
- Debug common queue bugs: overflow, underflow, modulo mistakes, NULL dereferences, memory leaks
- Choose the right variant: circular queue, priority queue, or deque
- Recognize queue-shaped problems in competitive programming and interviews

Keep Practicing! Queues are the backbone of BFS, scheduling, and buffer management.